# Mathematical and C++20 DSP Specification for Flagship Audiophile Acoustic Processors

## Non-Linear Vacuum Tube Emulation and Output Transformer Magnetics

### Continuous-Time Physics and Equations of Koren and Norman Tube Models

The vacuum tube triode and pentode stages are modeled using the phenomenological formulations developed by Norman Koren. In high-end acoustic emulation, these models supersede idealized Child-Langmuir space-charge formulations by addressing the soft transition into cutoff, modeling continuous grid conduction, and reproducing electrostatic plate-knee saturation dynamics without introducing mathematical discontinuities.

The physical tube structure comprises an electron-emitting heated cathode, an electrostatic control grid that regulates electron stream density, and an anode plate that collects accelerated electrons. In a triode circuit with plate voltage v_p, grid voltage v_g, and cathode voltage v_k, the operational potential differences relative to cathode are defined as v_{pk} = v_p - v_k and v_{gk} = v_g - v_k. The effective controlling potential E_1(v_{pk}, v_{gk}) governs the electrostatic field influencing the space charge near the cathode:

E_1(v_{pk}, v_{gk}) =[span_1](start_span)[span_1](end_span) \frac{v_{pk}}{k_P} \ln\left[1 + \exp\left(k_P \left(\frac{1}{\mu} + \frac{v_{gk}}{\sqrt{K_{VB} + v_{pk}^2}}\right)\right)\right]

The resulting continuous plate current I_p is expressed as a non-linear power function:

I_p(v_{pk}, v_{gk}) = \begin{cases} \dfrac{E_1(v_{pk}, v_{gk})^{x}}{k_{G1}}, & E_1 > 0 \\ 0, & E_1 \le 0 \end{cases}

In a pentode, an accelerating screen grid (g_2) and a suppressor grid (g_3, tied to cathode) decouple the plate voltage from cathode field control, transforming the tube into a voltage-controlled current source with high internal plate resistance. With screen-to-cathode potential v_{g2k} = v_{g2} - v_k, the primary controlling field E_{1,\text{pent}} is determined almost exclusively by the screen grid and control grid, while an arctangent function captures plate-knee electron collection efficiency:

E_{1,\text{pent}}(v_{[span_3](start_span)[span_3](end_span)g2k}, v_{gk}) = \frac{v_{g2k}}{k_P} \ln\left[1 + \exp\left(k_P \left(\frac{1}{\mu} + \frac{v_{gk}}{v_{g2k}}\right)\right)\right] I_p(v_{pk}, v_{gk}, v_{g2k}) = \begin{cases} \dfrac{E_{1,\text{pent}}^{x}}{k_{G1}} \cdot \arctan\left(\dfrac{v_{pk}}{K_{VB}}\right), & E_{1,\text{pent}} > 0 \\ 0, & E_{1,\text{pent}} \le 0 \end{cases}

Screen grid current I_{g2} diverts electrons from the plate at low plate voltages, maintaining space-charge continuity according to:

I_{g2}(v_{pk}, v_{gk}, v_{g2k}) = \frac{E_{1,\text{pent}}^{x}}{k_{G2}} \left[1 - \frac{2}{\pi}\arctan\left(\frac{v_{pk}}{K_{VB}}\right)\right]

| Parameter | 12AX7 / ECC83 (Triode) | EL34 / 6CA7 (Pentode) | Physical Interpretation |
|---|---|---|---|
| \mu | 100.0 | 10.5 (triode mode \mu) | Amplification factor of the control grid |
| k_P | 600.0 | 26[span_4](start_span)[span_4](end_span)0.0 | Exponential onset knee sharpness factor |
| K_{VB} | 300.0\tex[span_5](start_span)[span_5](end_span)t{ V}^2 | 24.0\text{ V} | Plate voltage knee inflection coefficient |
| k_{G1} | 1060.0 | 1200.0 | Plate transconductance scaling denominator |
| x | 1.40 | 1.35 | Exponent governing deviation from 3/2 Child-Langmuir law |

When mapped into Wave Digital Filters (WDF), the tube models behave as non-linear multi-port resistors whose port voltages v_k and currents i_k map to wave variables a_k = v_k + R_k i_k and b_k = v_k - R_k i_k. Because the Koren plate equations express current I_p implicitly in terms of terminal potentials, real-time WDF implementations require a multidimensional Newton-Raphson solver or precomputed two-dimensional lookup tables (LUT) with bivariate cubic Hermite interpolation.

## Memoryless Asymmetric Hyperbolic Waveshaper

When processing constraints prevent the deployment of iterative implicit solvers, a memoryless asymmetric hyperbolic transfer function accurately reproduces the static transfer characteristics and harmonic spectra of a single-ended triode stage:
$$f(x) = \frac{\tanh(\alpha x + \beta) - \tanh(\beta)}{1 - \tanh(\beta)} + \gamma \left( \sqrt{1 + (\alpha x)^2} - 1 \right)$$
In this formulation, \alpha dictates the input drive hardness, \beta sets the internal DC operating bias asymmetry (generating second- and fourth-order even harmonics), and \gamma introduces dynamic quadratic curvature that models load-line compression.

## Output Transformer Core Saturation and Jiles-Atherton Hysteresis

Audio output transformers exhibit pronounced core saturation, remanent magnetization, and hysteresis at low frequencies, particularly below 20\text{ Hz} where the primary magnetizing reactance decreases and magnetic flux density approaches the saturation limit B_{\text{sat}}.
The Jiles-Atherton differential model parameterizes the magnetic field intensity $H(t) = \frac{N_p \cdot i_p(t)}{l_m}$ and core magnetization M(t):
The effective magnetic field H_e, accounting for inter-domain coupling \alpha_m, is given by:
$$H_e = H + \alpha_m M$$
The anhysteretic magnetization M_{\text{an}} is calculated using the modified Langevin function:
$$M_{\text{an}} = M_s \left[ \coth\left(\frac{H_e}{a}\right) - \frac{a}{H_e} \right]$$
The irreversible magnetization differential equation accounts for domain wall pinning:
$$\frac{dM_{\text{irr}}}{dH} = \frac{M_{\text{an}} - M_{\text{irr}}}{k_m \delta - \alpha_m (M_{\text{an}} - M_{\text{irr}})}$$
where $\delta = \operatorname{sgn}\left(\frac{dH}{dt}\right)$ enforces thermodynamic consistency.

Reversible elastic domain wall bowing produces an additional magnetization component: $M_{\text{rev}} = c_m (M_{\text{an}} - M_{\text{irr}})$
Total core magnetization is $M = M_{\text{irr}} + M_{\text{rev}}$. In discrete time with sampling interval $T_s = \frac{1}{f_s}$ and $\Delta H[n] = H[n] - H[n-1]$, the state update equation becomes:
$M_{\text{irr}}[n] = M_{\text{irr}}[n-1] + \Delta H[n] \left[ \frac{M_{\text{an}}[n-1] - M_{\text{irr}}[n-1]}{k_m \operatorname{sgn}(\Delta H[n]) - \alpha_m (M_{\text{an}}[n-1] - M_{\text{irr}}[n-1])} \right]$
The induced secondary voltage $v_s[n]$ across secondary turns $N_s$ with core area $A_{\text{core}}$ is derived via the discrete backward difference of magnetic flux density $B = \mu_0(H + M)$:
$v_s[n] = -N_s A_{\text{core}} \mu_0 \left( \frac{H[n] - H[n-1]}{T_s} + \frac{M[n] - M[n-1]}{T_s} \right)$
As $H \to \infty$, the differential permeability $\frac{dM}{dH} \to 0$, causing the effective primary inductance $L_m \approx \mu_0 N_p^2 \frac{A_{\text{core}}}{l_m} \left(1 + \frac{dM}{dH}\right)$ to collapse toward air-core levels. This physical behavior produces odd-harmonic generation, dynamic low-frequency shelf loss, and progressive phase lead below $20\text{ Hz}$.

## Polyphase Multirate Half-Band Filtering Pipeline

Nonlinear harmonic generation expands signal bandwidth, demanding an $8\times$ oversampling rate ($352.8\text{ kHz}$ or $384\text{ kHz}$) to prevent aliasing. In a polyphase implementation, the prototype transfer function $H(z)$ is decomposed into two polyphase components:
$H(z) = E_0(z^2) + z^{-1} E_1(z^2)$
For 1:2 interpolation, input samples $x[n]$ are fed into the polyphase branches $E_0(z)$ and $E_1(z)$, and their outputs are interleaved to construct the upsampled stream $y[m]$. Conversely, for 2:1 decimation, the input samples are split into even and odd indices, filtered through the polyphase components, and summed to yield the baseband signal. In an equiripple linear-phase half-band FIR filter, every even coefficient relative to the center tap is zero ($h[2m] = 0$ for $m \neq 0$), yielding $E_0(z) = h[M] = 0.5$ and reducing arithmetic operations by half.

| Tap Index k | Coefficient h[k] (23-Tap Half-Band FIR) | Tap Index k | Coefficient h[k] (23-Tap Half-Band FIR) |
|---|---|---|---|
| h[0], h[22] | -0.0128156194 | h[6], h[16] | +0.0527391897 |
| h[1], h[21] | 0.0000000000 | h[7], h[15] | 0.0000000000 |
| h[2], h[20] | +0.0177891106 | h[8], h[14] | -0.1000563092 |
| h[3], h[19] | 0.0000000000 | h[9], h[13] | 0.0000000000 |
| h[4], h[18] | -0.0306675072 | h[10], h[12] | +0.3198452013 |
| h[5], h[17] | 0.0000000000 | h[11] (Center) | +0.5063318686 |

For low-latency execution, a 5th-order two-path allpass IIR halfband filter is constructed:
$H(z) = \frac{1}{2} \left[ A_0(z^2) + z^{-1} A_1(z^2) \right] A_0(z) = \frac{\alpha_0 + z^{-1}}{1 + \alpha_0 z^{-1}}, \quad A_1(z) = \frac{\alpha_1 + z^{-1}}{1 + \alpha_1 z^{-1}}$
The optimal elliptic coefficients providing steep transition attenuation are $\alpha_0 = 0.1413486744$ and $\alpha_1 = 0.5899948201$.

# Virtual R2R Ladder and Non-Oversampling (NOS) DAC Modeling

## Staircase Aperture Distortion (Zero-Order Hold)

A Non-Oversampling (NOS) DAC updates its analog output at baseband frequency f_s (44.1\text{ kHz} or 48\text{ kHz}) and maintains that level over sample duration T = \frac{1}{f_s}. The output signal represents a convolution of discrete impulse values with a rectangular pulse p(t) = \operatorname{rect}\left(\frac{t - T/2}{T}\right).
Taking the continuous Fourier transform yields the Zero-Order Hold (ZOH) aperture transfer function:
H_{\text{ZOH}}(f) = \mathcal{F}\{p(t)\} = T \cdot e^{-j \pi f / f_s} \cdot \frac{\sin(\pi f / f_s)}{\pi f / f_s} = T \cdot e^{-j \pi f / f_s} \cdot \operatorname{sinc}\left(\frac{f}{f_s}\right)
The continuous magnitude response attenuates upper baseband audio frequencies:
|H_{\text{ZOH}}(f)| = \left| \frac{\sin(\pi f / f_s)}{\pi f / f_s} \right|
At the Nyquist frequency (f = \frac{f_s}{2}), the transfer function drops to:
|H_{\text{ZOH}}(f_s/2)| = \frac{\sin(\pi / 2)}{\pi / 2} = \frac{2}{\pi} \approx 0.6366198 \quad (-3.9224\text{ dB})
Unfiltered mirror images fold back across integer multiples of the sampling frequency at k f_s \pm f (k \in \{1, 2, 3, \dots\}), interacting with downstream amplifier input stages and generating intermodulation products.

## Sinc Roll-Off Equalization Filter Design

To restore flat frequency response up to 0.45 f_s, a linear-phase inverse-sinc FIR pre-emphasis filter C(z) is derived by least-squares fitting against C(e^{j\omega}) \approx \operatorname{sinc}^{-1}\left(\frac{\omega}{2\pi}\right):

| Tap Index k | Filter Coefficient c[k] (5-Tap Inverse-Sinc FIR) |
|---|---|
| c[0], c[4] | +0.024171 |
| c[1], c[3] | -0.096277 |
| c[2] (Center) | +1.155429 |

The Direct Form FIR difference equation is implemented as:

y_{\text{comp}}[n] = \sum_{k=0}^{4} c[k] \, x[n-k]

## Static Bit-Weight Mismatch, Thermal Drift, and Dynamic Glitches

In an ideal 24-bit binary-weighted R2R ladder DAC, the output voltage represents the radix-2 summation of input bits b_k \in \{0, 1\}:
V_{\text{ideal}} = V_{\text{ref}} \sum_{k=1}^{24} b_k 2^{-k}
In a physical R2R network, reference currents divide through a repeating topology of series resistors of nominal value R and shunt resistors of nominal value 2R. Discrete thin-film fabrication limits introduce stochastic resistance deviations \epsilon_{R,k}, \epsilon_{2R,k} \sim \mathcal{N}(0, \sigma_R^2). The effective bit weight for bit k diverges from nominal binary weighting:
w_k = 2^{-k} (1 + \delta_k)
where \delta_k denotes the cumulative error derived from ladder nodal admittance equations. A discrepancy in the Most Significant Bit (MSB, k=1) produces a Differential Non-Linearity (DNL) error during the mid-scale transition (0111\dots1 \to 1000\dots0):
\text{DNL}_{\text{mid}} = \frac{V(1000\dots0) - V(0111\dots1)}{V_{\text{LSB}}} - 1 = \frac{\delta_1 - \sum_{k=2}^{24} \delta_k 2^{-(k-1)}}{2^{-24}}

Integral Non-Linearity (INL) represents the running sum:
$$\text{INL}[D] = \sum_{j=0}^{D} \text{DNL}[j]$$
Resistor self-heating alters ladder ratios during sustained high-amplitude passages in accordance with the temperature coefficient of resistance (TCR, ppm/°C). Thermal state $T_k[n]$ tracks power dissipation with time constant $\tau_{\text{th}} \approx 10\text{ ms}$:
$$T_k[n] = T_k[n-1] + \frac{T_s}{\tau_{\text{th}}} \left( R_k \cdot i_k^2[n] - \kappa_{\text{diss}} (T_k[n-1] - T_{\text{ambient}}) \right) \delta_k[n] = \delta_{k, \text{static}} + \gamma_{\text{TCR}} \cdot (T_k[n] - T_{\text{ambient}})$$
Differential switch propagation delays $\Delta t_k$ and parasitic charge injections $Q_{\text{inj}, k}$ produce transient impulses during code transitions, exciting a damped high-frequency oscillation:

$$g[n] = \sum_{k=1}^{24} \vert{}b_k[n] - b_k[n-1]\vert{} \cdot A_{\text{glitch}, k} \cdot e^{-\zeta_{\text{gl}} \omega_{\text{gl}} t_{\text{trans}}} \cos(\omega_{\text{gl}} \sqrt{1 - \zeta_{\text{gl}}^2} \, t_{\text{trans}})$$

# Sony-Style Analog Processors

## Analog Amplifier Low-Frequency DC Phase Linearizer

Discrete analog power amplifiers employ AC-coupling capacitors in signal paths and negative feedback loops, introducing a multi-pole high-pass response that rotates phase above [span_29](start_span)[span_29](end_span)[span_32](start_span)[span_32](end_span)[span_35](start_span)[span_35](end_span)$0\text{ Hz}$. Conversely, DC-coupled digital architectures (such as Sony S-Master HX) maintain flat phase down to DC. The DC Phase Linearizer reintroduces the low-frequency phase shifts characteristic of classic analog amplifier topologies to alter bass transient decay without altering magnitude response.
Phase rotation is synthesized through a second-order digital allpass biquad:
$$H_{\text{AP}}(z) = \frac{a_2 + a_1 z^{-1} + z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$
Given continuous resonant frequency $\omega_0 = 2\pi f_0$ and damping factor $\zeta = \frac{1}{2Q}$, applying the Bilinear Transform with prewarping ($K = \tan\left(\frac{\pi f_0}{f_s}\right)$) yields the normalized filter coefficients:
$$a_0 = K^2 + 2\zeta K + 1 \quad a_1 = \frac{2(K^2 - 1)}{a_0} \\ a_2 = \frac{K^2 - 2\zeta K + 1}{a_0}$$
The normalized allpass numerator coefficients are $b_0 = a_2$, $b_1 = a_1$, and $b_2 = 1.0$.
Sony DC Phase Linearizer implementations provide six presets divided into Type A and Type B topologies. Type A presets emulate DC-servo solid-state amplifiers, introducing moderate phase lead to deliver tight, highly damped bass transients. Type B presets reproduce transformer- and capacitor-coupled tube circuits, introducing extended low-frequency phase shift below $60\text{ Hz}$ that prolongs bass decay and enriches low-frequency warmth.

| Preset Mode | Pole Frequency $f_0$ | Quality Factor Q | Phase Shift $\phi(20\text{ Hz})$ | Acoustic Signature |
|---|---|---|---|---|
| **Type A - Low** | $18.0\text{ Hz}$ | 0.55 | $+18.5^\circ$ | Clean sub-bass definition, controlled driver excursion |
| **Type A - Standard** | $32.0\text{ Hz}$ | 0.707 | $+38.2^\circ$ | Balanced solid-state low-frequency |

| Preset Mode | Pole Frequency $f_0$ | Quality Factor Q | Phase Shift $\phi(20\text{ Hz})$ | Acoustic Signature |
|---|---|---|---|---|
| | | | | impact |
| **Type A - High** | $55.0\text{ Hz}$ | 0.85 | $+64.1^\circ$ | Pronounced mid-bass focus, short decay times |
| **Type B - Low** | $22.0\text{ Hz}$ | 0.45 | $+31.0^\circ$ | Extended sub-bass resonance, subtle bloom |
| **Type B - Standard** | $42.0\text{ Hz}$ | 0.60 | $+62.4^\circ$ | Classic push-pull tube amplifier envelope |
| **Type B - High** | $75.0\text{ Hz}$ | 0.90 | $+105.3^\circ$ | Vintage transformer-coupled acoustic character |

## Turntable Mechanical Dynamics and Vinyl Physical Artifacts

Emulating vinyl playback requires physical modeling of the tonearm assembly, electro-mechanical transducer crosstalk, and groove friction mechanics.

A phono playback system behaves as a coupled mechanical assembly. The tonearm mass $M_{\text{eff}}$ pivots on bearings, suspending the phono cartridge over the spinning record. At the contact point, the diamond stylus tracks groove modulations, transferring forces through a compliant elastomer cantilever suspension characterized by compliance $C_{\text{cart}}$.

The effective mass of the tonearm and cartridge ($M_{\text{eff}} \approx 14\text{ g}$) interacting with the cantilever suspension compliance ($C_{\text{cart}} \approx 15 \times 10^{-6}\text{ cm/dyne}$) forms a second-order mass-spring-damper resonance:

$$f_{\text{res}} = \frac{1}{2\pi \sqrt{M_{\text{eff}} \cdot C_{\text{cart}}}}$$

With typical physical parameters, this resonance settles near $11.0\text{ Hz}$. A second-order peak biquad with center frequency $f_0 = 11.0\text{ Hz}$ and $Q = 3.5$ models this mechanical behavior. The resonator is driven by simulated low-frequency record warp noise modeled as a pink-noise process filtered at the platter rotation rate ($33\frac{1}{3}\text{ RPM} \implies 0.555\text{ Hz}$).

Mutual inductance between stereo pickup coils and mechanical compliance coupling across the cantilever pivot induce crosstalk between channels:

$$L_{\text{out}}(t) = L_{\text{in}}(t) + \kappa_{\text{mid}} R_{\text{in}}(t) + \tau_{\text{hf}} \frac{d R_{\text{in}}(t)}{dt} \quad R_{\text{out}}(t) = R_{\text{in}}(t) + \kappa_{\text{mid}} L_{\text{in}}(t) + \tau_{\text{hf}} \frac{d L_{\text{in}}(t)}{dt}$$

where $\kappa_{\text{mid}} \approx 0.0316$ ($-30\text{ dB}$ midband crosstalk) and $\tau_{\text{hf}}$ models inductive coupling rising at $+6\text{ dB/octave}$ above $4\text{ kHz}$.

Because a spherical or elliptical replay stylus has a finite contact radius $R_{\text{stylus}}$, it cannot trace the profile cut by a triangular lathe chisel. For a lateral sinusoidal groove modulation $y(x) = A \sin(k x)$, the stylus tip experiences tracing distortion (Pinch Effect):

$$y_{\text{stylus}}(t) \approx y(t) - \frac{R_{\text{stylus}}^2}{4 v_{\text{groove}}^2} \left( \frac{dy}{dt} \right)^2 \frac{d^2 y}{dt^2}$$

This nonlinearity generates second-order distortion on vertical (difference L-R) signals and

dominant third-order distortion on lateral (sum L+R) signals. Dry boundary friction between diamond and vinyl adds microscopic chatter modeled as signal-modulated surface drag noise:

e_{\text{surface}}[n] = \sigma_{\text{drag}} \cdot \left(1 + \mu_{\text{mod}} \vert{}x[n]\vert{}\right) \cdot \eta_{\text{Gaussian}}[n]

# Multi-Stage DSD/PCM Decimation and SDM Reconstruction

## Multirate DSD-to-PCM Decimation Architecture

Direct Stream Digital (DSD) uses 1-bit Delta-Sigma modulation at multiples of 44.1\text{ kHz}: DSD64 (2.8224\text{ MHz}), DSD128 (5.6448\text{ MHz}), and DSD256 (11.2896\text{ MHz}). Decimating these streams to 64-bit float PCM at 352.8\text{ kHz} or 176.4\text{ kHz} requires sharp attenuation of shaped high-frequency quantization noise.
The decimation pipeline converts the high-rate 1-bit DSD stream to baseband PCM through three successive filtering stages:

- A 5th-order Cascaded Integrator-Comb (CIC) decimator operates on the high-rate DSD stream, executing an initial decimation by a factor of 16 down to 705.6\text{ kHz}.
- A CIC passband droop compensation FIR filter restores flat frequency response across the passband.
- A half-band FIR decimation stage downsamples the intermediate signal by a factor of 2 to 352.8\text{ kHz}.
- A final reconstruction FIR filter provides sharp ultrasonic stopband rejection with selectable cutoff profiles down to 176.4\text{ kHz} or 352.8\text{ kHz}.

For decimation factor R_1 = 16 and differential delay M = 1, the 5th-order CIC transfer function is:
H_{\text{CIC}}(z) = \left( \frac{1 - z^{-16}}{1 - z^{-1}} \right)^5 = \left( \sum_{k=0}^{15} z^{-k} \right)^5
Using the Hogenauer non-recursive architecture, the filter executes via five discrete integrators running at the high input rate f_{\text{DSD}}, followed by 16\times downsampling and five comb stages operating at f_{\text{int}} = \frac{f_{\text{DSD}}}{16}:
y_{i}[n] = y_{i}[n-1] + x_{i}[n] \quad (i = 1 \dots 5) y_{c}[m] = x_{c}[m] - x_{c}[m-1] \quad (c = 1 \dots 5)
Register growth requires bit capacity B_{\text{max}} = B_{\text{in}} + 5 \log_2(16) = 1 + 5(4) = 21\text{ bits}, which fits into the 53-bit mantissa of 64-bit IEEE double-precision floats.

| Filter Profile | Passband Ripple | Stopband Cutoff | Stopband Attenuation | Objective |
|---|---|---|---|---|
| **Strict Brickwall** | < 0.00005\text{ dB} | 24.0\text{ kHz} | > 120\text{ dB} | Suppression of ultrasonic DSD noise |
| **Gentle Roll-off** | < 0.01\text{ dB} | 30.0\text{ kHz} | > 90\text{ dB} | Extended phase linearity and transient preservation |
| **Wideband High-Res** | < 0.05\text{ dB} | 47.0\text{ kHz} | > 60\text{ dB} | Extended group delay linearity up |

| Filter Profile | Passband Ripple | Stopband Cutoff | Stopband Attenuation | Objective |
|---|---|---|---|---|
| | | | | to 40\t[span_66](start_span)[span_66](end_span)[span_68](start_span)[span_68](end_span)ext{ kHz} |

## 5th-Order Noise-Shaped Delta-Sigma Modulator (CIFF)

Converting 64-bit PCM back to 1-bit DSD256 (11.2896\text{ MHz}) uses a Cascade of Integrators with FeedForward (CIFF) architecture with optimal Noise Transfer Function (NTF) zeros on the unit circle.

In this structure, the incoming oversampled PCM signal passes into an error summer followed by five cascaded delay-integrators. Feedforward paths scale each integrator state to drive a 1-bit comparator, while negative local resonator loops place transmission zeros at specific frequencies across the audio band.

\text{NTF}(z) = \frac{(z - 1) \prod_{k=1}^{2} \left( z^2 - 2\cos(\omega_k) z + 1 \right)}{D(z)}

At f_{\text{DSD}} = 11.2896\text{ MHz} and audio bandwidth f_B = 24.0\text{ kHz}, the oversampling ratio is \text{OSR} = 256. Normalized bandwidth is \omega_B = \frac{2\pi \cdot 24000}{11289600} \approx 0.0133571\text{ rad}. Placing the zeros according to Schreier's optimal localizer roots minimizes total in-band quantization noise:

\omega_1 = 0.53846931 \cdot \omega_B \approx 7.192395 \times 10^{-3}\text{ rad} \omega_2 = 0.90617985 \cdot \omega_B \approx 1.210395 \times 10^{-2}\text{ rad}

The local feedback resonator coefficients are derived directly from the zero frequencies:

g_1 = 2\left(1 - \cos(\omega_1)\right) \approx 5.173032 \times 10^{-5} g_2 = 2\left(1 - \cos(\omega_2)\right) \approx 1.465037 \times 10^{-4}

The denominator polynomial D(z) is synthesized using a 5th-order Butterworth high-pass filter designed to satisfy Lee's criterion for 1-bit loop stability:

\max_{\omega} \vert{}\text{NTF}(e^{j\omega})\vert{} = \Vert{}\text{NTF}\Vert{}_\infty \le 1.50

With state vector \mathbf{x}[n] = [x_1[n], x_2[n], x_3[n], x_4[n], x_5[n]]^T, input u[n], and 1-bit output y[n] = \operatorname{sgn}\left(\sum_{i=1}^5 c_i x_i[n] + c_0 u[n]\right) \in \{-1.0, +1.0\}, the state-space integrator update equations are:

x_1[n+1] = x_1[n] + b_1 (u[n] - y[n]) x_2[n+1] = x_2[n] + x_1[n+1] - g_1 x_3[n] x_3[n+1] = x_3[n] + x_2[n+1] x_4[n+1] = x_4[n] + x_3[n+1] - g_2 x_5[n] x_5[n+1] = x_5[n] + x_4[n+1]

| Feedback / Feedforward Tap | Numerical Value | Feedback / Feedforward Tap | Numerical Value |
|---|---|---|---|
| b_1 | 0.461284 | c_3 | 0.089473 |
| c_0 | 1.000000 | c_4 | 0.012541 |
| c_1 | 0.628491 | c_5 | 0.001128 |
| c_2 | 0.274158 | g_1, g_2 | 5.173032 \times 10^{-5}, 1.465037 \times 10^{-4} |

# ISO 226:2023 Dynamic Equal-Loudness and Virtual

# Sub-Bass

## ISO 226:2023 Parameter Formulation and Shelving Filter Fitting

The human auditory system exhibits non-linear, frequency-dependent sensitivity that flattens at high sound pressure levels and rolls off steeply in the bass and treble registers at lower listening volumes. The ISO 226:2023 standard updates auditory threshold values T_f (incorporating reference zero shifts from ISO 389-7:2019) and refines the perceptual power exponents \alpha_f. The continuous relationship mapping sound pressure level L_p(f, L_N) (in dB SPL) to loudness level L_N (in phons) is defined as:

$$L_p(f, L_N) = \frac{10}{\alpha_f} \log_{10} \left[ 4 \cdot 10^{-10} \left( 10^{\frac{L_N}{10}} - 10^{\frac{T_r}{10}} \right) + 10^{\frac{\alpha_f (T_f + L_U)}{10}} \right] - L_U$$

where $T_r = 0.0\text{ phon}$, T_f represents the absolute threshold of hearing at frequency f, L_U denotes the normalized free-field acoustic transfer function, and \alp[span_79](start_span)[span_79](end_span)ha_f represents the auditory loudness exponent.

| Frequency f (Hz) | Loudness Exponent \alpha_f | Transfer Magnitude L_U (dB) | Absolute Threshold T_f (dB SPL) |
|---|---|---|---|
| **20** | 0.532 | -31.6 | 78.1 |
| **50** | 0.375 | -13.8 | 44.0 |
| **100** | 0.298 | -5.9 | 26.5 |
| **500** | 0.215 | 0.0 | 4.4 |
| **1000** | 0.200 | 0.0 | 2.4 |
| **3150** | 0.210 | +3.5 | -3.0 |
| **10000** | 0.254 | -2.5 | 13.2 |

When playback volume drops from reference calibration $L_{\text{ref}} = 80\text{ phon}$ to lower levels, the required equalization curve \Delta G(f, L[span_84](start_span)[span_84](end_span)[span_86](start_span)[span_86](end_span)_N) tracks the difference between the contours:

$$\Delta G(f, L_N) = \left[ L_p(f, L_N) - L_p(1000, L_N) \right] - \left[ L_p(f, L_{\text{ref}}) - L_p(1000, L_{\text{ref}}) \right]$$

This differential contour is tracked using three parametric biquad sections linked to the attenuation level $A_{\text{dB}} \in [-60\text{ dB}, 0\text{ dB}]$:

- A low-frequency shelving biquad centered at $f_{0,\text{low}} = 85.0\text{ Hz}$ compensates for bass perception loss.
- A high-frequency shelving biquad centered at $f_{0,\text{high}} = 9000.0\text{ Hz}$ compensates for high-frequency perceptual loss.
- A parametric peaking biquad centered at $f_{0,\text{mid}} = 3200.0\text{ Hz}$ compensates for varying ear-canal resonance sensitivity.

For the low-frequency shelf ($f_{0,\text{low}} = 85.0\text{ Hz}$, $S = 0.707$), the gain tracks volume attenuation:

$$G_{\text{low}}(A_{\text{dB}}) = -0.38 \cdot A_{\text{dB}} \quad (+22.8\text{ dB boost at } -60\text{ dBFS})$$

With $A = 10^{G_{\text{low}} / 40}$, $\omega_0 = \frac{2\pi f_0}{f_s}$, and $\alpha = \frac{\sin(\omega_0)}{2} \sqrt{\left(A + \frac{1}{A}\right)\left(\frac{1}{S} - 1\right) + 2}$, the normalized coefficients are:

$$b_0 = A \left[ (A+1) - (A-1)\cos(\omega_0) + 2\sqrt{A}\alpha \right] \quad b_1 = 2A \left[ (A-1) -$$

(A+1)\cos(\omega_0) \right] \\ b_2 = A \left[ (A+1) - (A-1)\cos(\omega_0) - 2\sqrt{A}\alpha \right] \\ a_0 = (A+1) + (A-1)\cos(\omega_0) + 2\sqrt{A}\alpha \\ a_1 = -2 \left[ (A-1) + (A+1)\cos(\omega_0) \right] a_2 = (A+1) + (A-1)\cos(\omega_0) - 2\sqrt{A}\alpha
The high-shelf filter (f_{0,\text{high}} = 9000.0\text{ Hz}, S = 0.707) tracks G_{\text{high}}(A_{\text{dB}}) = -0.11 \cdot A_{\text{dB}} (+6.6\text{ dB} at -60\text{ dBFS}). The ear-canal peaking filter (f_{0,\text{mid}} = 3200.0\text{ Hz}, Q = 1.4) tracks G_{\text{mid}}(A_{\text{dB}}) = +0.05 \cdot A_{\text{dB}} (-3.0\text{ dB} attenuation at -60\text{ dBFS}).

## Phase-Coherent "Missing Fundamental" Virtual Sub-Bass Generator

Micro-transducers in portable Bluetooth systems cannot reproduce acoustic energy below their mechanical cutoff frequency (f_{\text{cutoff}} \approx 50\text{--}70\text{ Hz}). Driving drivers below resonance causes voice-coil distortion and mechanical bottoming without producing audible fundamental output. The "Missing Fundamental" algorithm suppresses inaudible sub-bass (20\text{--}45\text{ Hz}) and reconstructs the perceived low-pitch sensation by synthesizing phase-aligned second and third harmonics in the audible passband (40\text{--}135\text{ Hz}).
In this signal path, the primary input signal splits into a clean path and a sub-bass synthesis path. The clean path traverses an allpass delay line that compensates for the phase delay introduced by the harmonic synthesis chain. In the parallel synthesis path:

- A 4th-order Linkwitz-Riley low-pass filter (f_c = 45.0\text{ Hz}) isolates the deep sub-bass fundamental.
- A complementary 4th-order Linkwitz-Riley high-pass filter removes the inaudible sub-bass from the main clean signal path to preserve amplifier headroom.
- The isolated fundamental drives a Chebyshev polynomial non-linear generator, synthesizing second- and third-order harmonics without intermodulation sidebands.
- A bandpass filter (45\text{--}140\text{ Hz}) removes residual DC offsets and high-frequency products.
- The synthesized harmonics are recombined with the delayed clean signal.

Orthogonal Chebyshev polynomials of the first kind T_k(x) synthesize the harmonics from normalized input v_{\text{in}} = \operatorname{clip}(x_{\text{sub}} / V_{\text{thresh}}, -1.0, 1.0): 2\text{nd Harmonic: } T_2(v_{\text{in}}) = 2 v_{\text{in}}^2 - 1 3\text{rd Harmonic: } T_3(v_{\text{in}}) = 4 v_{\text{in}}^3 - 3 v_{\text{in}}
Because T_2(\cos \omega t) = \cos(2\omega t) and T_3(\cos \omega t) = \cos(3\omega t), the polynomial expansion generates clean harmonic multiples without intermodulation distortion sidebands. The combined harmonic excitation signal is:
y_{\text{harm}}[n] = k_2 \cdot T_2(v_{\text{in}}[n]) + k_3 \cdot T_3(v_{\text{in}}[n])
where k_2 \approx 0.65 and k_3 \approx 0.35.
To prevent phase cancellation and ensure that the synthesized harmonics fuse perceptually into a unified bass note rather than a disjoint fuzz tone, the clean signal path is delayed by \Delta n samples to align with the extraction filter group delay:

\tau_{\text{LPF}} = -\left. \frac{d\phi_{\text{LPF}}(\omega)}{d\omega} \right\vert{}_{\omega=2\pi \cdot 35\text{ Hz}} \approx \frac{2\sqrt{2}}{\omega_c} \text{ seconds} \Delta n = \operatorname{round}(\tau_{\text{LPF}} \cdot f_s)

# C++20 Flagship Audiophile Acoustic Processor

```
#ifndef AUDIOPHILE_DSP_PIPELINE_HPP
#define AUDIOPHILE_DSP_PIPELINE_HPP

#include <cmath>
#include <numbers>
#include <array>
#include <span>
#include <algorithm>
#include <cstdint>

namespace Audiophile::DSP {

using std::numbers::pi;

//
================================================================================
======
// 1. Transposed Direct-Form II Biquad Filter Engine
//
================================================================================
======
class Biquad {
public:
    struct Coeffs {
        double b0{1.0}, b1{0.0}, b2{0.0};
        double a1{0.0}, a2{0.0};
    };

    constexpr Biquad() noexcept = default;

    void setCoefficients(const Coeffs& c) noexcept {
        coeffs_ = c;
    }

    void reset() noexcept {
        s1_ = 0.0;
        s2_ = 0.0;
    }

    [[nodiscard]] inline double process(double in) noexcept {
        double out = in * coeffs_.b0 + s1_;
        s1_ = in * coeffs_.b1 - coeffs_.a1 * out + s2_;
        s2_ = in * coeffs_.b2 - coeffs_.a2 * out;
        return out;
    }
```

```
    static Coeffs makeAllpass(double sampleRate, double frequency,
double q) noexcept {
        const double w0 = 2.0 * pi * frequency / sampleRate;
        const double alpha = std::sin(w0) / (2.0 * q);
        const double a0 = 1.0 + alpha;
        return {
            .b0 = (1.0 - alpha) / a0,
            .b1 = (-2.0 * std::cos(w0)) / a0,
            .b2 = (1.0 + alpha) / a0,
            .a1 = (-2.0 * std::cos(w0)) / a0,
            .a2 = (1.0 - alpha) / a0
        };
    }

    static Coeffs makeLowShelf(double sampleRate, double frequency,
double gainDb, double shelfSlope = 0.707) noexcept {
        const double A = std::pow(10.0, gainDb / 40.0);
        const double w0 = 2.0 * pi * frequency / sampleRate;
        const double cosw0 = std::cos(w0);
        const double sinw0 = std::sin(w0);
        const double alpha = sinw0 / 2.0 * std::sqrt((A + 1.0 / A) *
(1.0 / shelfSlope - 1.0) + 2.0);
        const double a0 = (A + 1.0) + (A - 1.0) * cosw0 + 2.0 *
std::sqrt(A) * alpha;

        return {
            .b0 = (A * ((A + 1.0) - (A - 1.0) * cosw0 + 2.0 *
std::sqrt(A) * alpha)) / a0,
            .b1 = (2.0 * A * ((A - 1.0) - (A + 1.0) * cosw0)) / a0,
            .b2 = (A * ((A + 1.0) - (A - 1.0) * cosw0 - 2.0 *
std::sqrt(A) * alpha)) / a0,
            .a1 = (-2.0 * ((A - 1.0) + (A + 1.0) * cosw0)) / a0,
            .a2 = ((A + 1.0) + (A - 1.0) * cosw0 - 2.0 * std::sqrt(A)
* alpha) / a0
        };
    }

    static Coeffs makeHighShelf(double sampleRate, double frequency,
double gainDb, double shelfSlope = 0.707) noexcept {
        const double A = std::pow(10.0, gainDb / 40.0);
        const double w0 = 2.0 * pi * frequency / sampleRate;
        const double cosw0 = std::cos(w0);
        const double sinw0 = std::sin(w0);
        const double alpha = sinw0 / 2.0 * std::sqrt((A + 1.0 / A) *
(1.0 / shelfSlope - 1.0) + 2.0);
        const double a0 = (A + 1.0) - (A - 1.0) * cosw0 + 2.0 *
std::sqrt(A) * alpha;
```

```
        return {
            .b0 = (A * ((A + 1.0) + (A - 1.0) * cosw0 + 2.0 *
std::sqrt(A) * alpha)) / a0,
            .b1 = (-2.0 * A * ((A - 1.0) + (A + 1.0) * cosw0)) / a0,
            .b2 = (A * ((A + 1.0) + (A - 1.0) * cosw0 - 2.0 *
std::sqrt(A) * alpha)) / a0,
            .a1 = (2.0 * ((A - 1.0) - (A + 1.0) * cosw0)) / a0,
            .a2 = ((A + 1.0) - (A - 1.0) * cosw0 - 2.0 * std::sqrt(A)
* alpha) / a0
        };
    }

    static Coeffs makePeak(double sampleRate, double frequency, double
gainDb, double q) noexcept {
        const double A = std::pow(10.0, gainDb / 40.0);
        const double w0 = 2.0 * pi * frequency / sampleRate;
        const double alpha = std::sin(w0) / (2.0 * q);
        const double a0 = 1.0 + alpha / A;

        return {
            .b0 = (1.0 + alpha * A) / a0,
            .b1 = (-2.0 * std::cos(w0)) / a0,
            .b2 = (1.0 - alpha * A) / a0,
            .a1 = (-2.0 * std::cos(w0)) / a0,
            .a2 = (1.0 - alpha / A) / a0
        };
    }

private:
    Coeffs coeffs_{};
    double s1_{0.0}, s2_{0.0};
};

//
================================================================================
======
// 2. Koren Triode & Pentode Tube Stage with JA Hysteresis
//
================================================================================
======
class VacuumTubeProcessor {
public:
    enum class TubeType { Triode12AX7, PentodeEL34 };

    struct JAParameters {
        double Ms{1.6e6};
        double a{1100.0};
```

```
        double alpha{1.6e-3};
        double k{400.0};
        double c{0.20};
    };

    explicit VacuumTubeProcessor(double sampleRate) noexcept
        : sampleRate_{sampleRate}, dt_{1.0 / sampleRate} {}

    void setModel(TubeType type) noexcept { tubeType_ = type; }
    void setDrive(double drive) noexcept { drive_ = std::max(0.1,
drive); }

    [[nodiscard]] double process(double in) noexcept {
        const double v_in = in * drive_ * 20.0;
        double ip = 0.0;

        if (tubeType_ == TubeType::Triode12AX7) {
            constexpr double mu = 100.0;
            constexpr double kp = 600.0;
            constexpr double kvb = 300.0;
            constexpr double kg1 = 1060.0;
            constexpr double x = 1.4;

            const double vpk = 250.0 - v_in * 15.0;
            const double vgk = -2.0 + v_in;

            const double inner = kp * (1.0 / mu + vgk / std::sqrt(kvb
+ vpk * vpk));
            const double e1 = (vpk / kp) *
std::log1p(std::exp(std::clamp(inner, -40.0, 40.0)));
            if (e1 > 0.0) {
                ip = std::pow(e1, x) / kg1;
            }
        } else {
            constexpr double mu = 10.5;
            constexpr double kp = 260.0;
            constexpr double kvb = 24.0;
            constexpr double kg1 = 1200.0;
            constexpr double x = 1.35;
            constexpr double vg2k = 250.0;

            const double vpk = 300.0 - v_in * 25.0;
            const double vgk = -13.5 + v_in;

            const double inner = kp * (1.0 / mu + vgk / vg2k);
            const double e1 = (vg2k / kp) *
std::log1p(std::exp(std::clamp(inner, -40.0, 40.0)));
            if (e1 > 0.0) {
```

```
                ip = (std::pow(e1, x) / kg1) * std::atan(std::max(0.0,
vpk) / kvb);
            }
        }

        const double audio_curr = (ip - 0.0012) * 2000.0;
        return processTransformerHysteresis(audio_curr);
    }

private:
    double processTransformerHysteresis(double current) noexcept {
        constexpr double Np = 1200.0;
        constexpr double lm = 0.08;
        const double H = (Np * current) / lm;
        const double dH = H - H_prev_;

        if (std::abs(dH) > 1e-12) {
            const double delta = (dH > 0.0) ? 1.0 : -1.0;
            const double He = H_prev_ + ja_.alpha * M_prev_;

            double Man = 0.0;
            if (std::abs(He) > 1e-6) {
                const double z = He / ja_.a;
                Man = (std::abs(z) > 30.0) ? (ja_.Ms * (z > 0.0 ? 1.0
: -1.0))
                                           : ja_.Ms * (1.0 /
std::tanh(z) - 1.0 / z);
            }

            const double denom = ja_.k * delta - ja_.alpha * (Man -
Mirr_prev_);
            double dMirr_dH = 0.0;
            if (std::abs(denom) > 1e-9 && ((Man - Mirr_prev_) * delta
> 0.0)) {
                dMirr_dH = (Man - Mirr_prev_) / denom;
            }

            const double Mirr = Mirr_prev_ + dMirr_dH * dH;
            const double M = Mirr + ja_.c * (Man - Mirr);

            constexpr double mu0 = 4.0 * pi * 1e-7;
            constexpr double Ns = 80.0;
            constexpr double Acore = 0.0006;
            const double v_sec = -Ns * Acore * mu0 * ((dH / dt_) + ((M
- M_prev_) / dt_)) * 0.005;

            H_prev_ = H;
            M_prev_ = M;
```

```
            Mirr_prev_ = Mirr;
            return std::clamp(v_sec, -2.5, 2.5);
        }

        return 0.0;
    }

    double sampleRate_;
    double dt_;
    TubeType tubeType_{TubeType::Triode12AX7};
    double drive_{1.0};
    JAParameters ja_{};
    double H_prev_{0.0};
    double M_prev_{0.0};
    double Mirr_prev_{0.0};
};

//
============================================================================
======
// 3. Virtual R2R Ladder and Non-Oversampling (NOS) DAC Simulator
//
============================================================================
======
class VirtualR2RDAC {
public:
    VirtualR2RDAC() noexcept {
        resetBitWeights();
    }

    void setMismatchSigma(double sigma) noexcept {
        uint64_t seed = 0x123456789ABCDEFULL;
        auto xorshift64 = [&seed]() {
            seed ^= seed << 13;
            seed ^= seed >> 7;
            seed ^= seed << 17;
            return seed;
        };

        for (size_t i = 0; i < 24; ++i) {
            double u1 = std::max(1e-15,
static_cast<double>(xorshift64()) / 18446744073709551615.0);
            double u2 = static_cast<double>(xorshift64()) /
18446744073709551615.0;
            double norm = std::sqrt(-2.0 * std::log(u1)) *
std::cos(2.0 * pi * u2);
            bitWeights_[i] = std::pow(2.0, -static_cast<double>(i +
1)) * (1.0 + norm * sigma);
```

```
        }
    }

    void resetBitWeights() noexcept {
        for (size_t i = 0; i < 24; ++i) {
            bitWeights_[i] = std::pow(2.0, -static_cast<double>(i +
1));
        }
    }

    void setSincCompensation(bool enable) noexcept {
        enableSincComp_ = enable;
    }

    [[nodiscard]] double process(double inputFloat) noexcept {
        const double clamped = std::clamp(inputFloat, -1.0, 1.0);
        const auto code = static_cast<int32_t>(clamped * 8388607.0);
        const auto unsignedCode = static_cast<uint32_t>(code +
8388608);

        double ladderVoltage = 0.0;
        uint32_t switchedBits = lastCode_ ^ unsignedCode;

        for (size_t bit = 0; bit < 24; ++bit) {
            if ((unsignedCode >> (23 - bit)) & 1U) {
                ladderVoltage += bitWeights_[bit];
            }
        }
        ladderVoltage = (ladderVoltage * 2.0) - 1.0;

        double glitch = 0.0;
        for (size_t bit = 0; bit < 24; ++bit) {
            if ((switchedBits >> (23 - bit)) & 1U) {
                glitch += (1.0 / static_cast<double>(bit + 1)) *
0.00025;
            }
        }
        lastCode_ = unsignedCode;

        glitchState_ = glitchState_ * 0.65 + glitch;
        double dacOut = ladderVoltage + glitchState_;

        if (enableSincComp_) {
            firBuf_[firIdx_] = dacOut;
            double comp = 0.0;
            for (size_t i = 0; i < 5; ++i) {
                size_t idx = (firIdx_ + 5 - i) % 5;
                comp += kSincFIR[i] * firBuf_[idx];
```

```
            }
            firIdx_ = (firIdx_ + 1) % 5;
            return comp;
        }

        return dacOut;
    }

private:
    static constexpr std::array<double, 5> kSincFIR{
        +0.024171, -0.096277, +1.155429, -0.096277, +0.024171
    };

    std::array<double, 24> bitWeights_{};
    std::array<double, 5> firBuf_{};
    size_t firIdx_{0};
    uint32_t lastCode_{0x800000};
    double glitchState_{0.0};
    bool enableSincComp_{true};
};

//
================================================================================
======
// 4. Sony-Style Analog Processors
//
================================================================================
======
class SonyAnalogProcessor {
public:
    enum class DCPhaseType {
        TypeA_Low,
        TypeA_Standard,
        TypeA_High,
        TypeB_Low,
        TypeB_Standard,
        TypeB_High,
        Off
    };

    struct VinylParameters {
        bool enable{true};
        double armResonanceGain{1.5};
        double crosstalk{0.0316};
        double surfaceDragNoise{0.0004};
    };

    explicit SonyAnalogProcessor(double sampleRate) noexcept
```

```
        : sampleRate_{sampleRate} {
        tonearmBiquad_.setCoefficients(Biquad::makePeak(sampleRate_,
11.0, 4.0, 3.5));
        setDCPhaseMode(DCPhaseType::TypeA_Standard);
    }

    void setDCPhaseMode(DCPhaseType mode) noexcept {
        phaseMode_ = mode;
        switch (mode) {
            case DCPhaseType::TypeA_Low:

allpassBiquad_.setCoefficients(Biquad::makeAllpass(sampleRate_, 18.0,
0.55));
                break;
            case DCPhaseType::TypeA_Standard:

allpassBiquad_.setCoefficients(Biquad::makeAllpass(sampleRate_, 32.0,
0.707));
                break;
            case DCPhaseType::TypeA_High:

allpassBiquad_.setCoefficients(Biquad::makeAllpass(sampleRate_, 55.0,
0.85));
                break;
            case DCPhaseType::TypeB_Low:

allpassBiquad_.setCoefficients(Biquad::makeAllpass(sampleRate_, 22.0,
0.45));
                break;
            case DCPhaseType::TypeB_Standard:

allpassBiquad_.setCoefficients(Biquad::makeAllpass(sampleRate_, 42.0,
0.60));
                break;
            case DCPhaseType::TypeB_High:

allpassBiquad_.setCoefficients(Biquad::makeAllpass(sampleRate_, 75.0,
0.90));
                break;
            case DCPhaseType::Off:
                break;
        }
    }

    void setVinylParams(const VinylParameters& params) noexcept {
        vinylParams_ = params;
    }
```

```
    void processStereo(double& left, double& right) noexcept {
        if (phaseMode_ != DCPhaseType::Off) {
            left = allpassBiquad_.process(left);
            right = allpassRightBiquad_.process(right);
        }

        if (vinylParams_.enable) {
            const double mono = 0.5 * (left + right);
            const double armRumble = tonearmBiquad_.process(mono) *
0.05 * vinylParams_.armResonanceGain;

            const double l_cross = left + vinylParams_.crosstalk *
right;
            const double r_cross = right + vinylParams_.crosstalk *
left;

            const double pinchL = l_cross - 0.012 * (l_cross * l_cross
* l_cross);
            const double pinchR = r_cross - 0.012 * (r_cross * r_cross
* r_cross);

            noiseSeed_ ^= noiseSeed_ << 13;
            noiseSeed_ ^= noiseSeed_ >> 7;
            noiseSeed_ ^= noiseSeed_ << 17;
            const double white = (static_cast<double>(noiseSeed_) /
9223372036854775807.0) - 1.0;
            const double surfNoise = white *
vinylParams_.surfaceDragNoise * (1.0 + 0.3 * std::abs(mono));

            left = pinchL + armRumble + surfNoise;
            right = pinchR + armRumble + surfNoise;
        }
    }

private:
    double sampleRate_;
    DCPhaseType phaseMode_{DCPhaseType::TypeA_Standard};
    Biquad allpassBiquad_{};
    Biquad allpassRightBiquad_{};
    Biquad tonearmBiquad_{};
    VinylParameters vinylParams_{};
    uint64_t noiseSeed_{0xCAFEBABEDEADBEEFULL};
};

//
================================================================
======
// 5. 5th-Order CIFF Noise-Shaped Delta-Sigma Modulator (PCM to
```

```
DSD256)
//
================================================================================
======
class DeltaSigmaModulator5th {
public:
    DeltaSigmaModulator5th() noexcept {
        reset();
    }

    void reset() noexcept {
        x_.fill(0.0);
    }

    [[nodiscard]] int8_t processSample(double u) noexcept {
        const double y_state = c_[0] * u + c_[1] * x_[0] + c_[2] *
x_[1]
                             + c_[3] * x_[2] + c_[4] * x_[3] + c_[5] *
x_[4];
        const double y = (y_state >= 0.0) ? 1.0 : -1.0;

        const double err = u - y;
        const double x0_next = x_[0] + b1_ * err;
        const double x1_next = x_[1] + x0_next - g1_ * x_[2];
        const double x2_next = x_[2] + x1_next;
        const double x3_next = x_[3] + x2_next - g2_ * x_[4];
        const double x4_next = x_[4] + x3_next;

        x_[0] = std::clamp(x0_next, -4.0, 4.0);
        x_[1] = std::clamp(x1_next, -4.0, 4.0);
        x_[2] = std::clamp(x2_next, -4.0, 4.0);
        x_[3] = std::clamp(x3_next, -4.0, 4.0);
        x_[4] = std::clamp(x4_next, -4.0, 4.0);

        return (y > 0.0) ? static_cast<int8_t>(1) :
static_cast<int8_t>(-1);
    }

private:
    static constexpr double b1_{0.461284};
    static constexpr double g1_{5.173032e-5};
    static constexpr double g2_{1.465037e-4};
    static constexpr std::array<double, 6> c_{
        1.000000, 0.628491, 0.274158, 0.089473, 0.012541, 0.001128
    };

    std::array<double, 5> x_{};
};
```

```
//
================================================================================
======
// 6. ISO 226:2023 Loudness and Virtual Sub-Bass Processor
//
================================================================================
======
class EqualLoudnessAndVirtualBass {
public:
    explicit EqualLoudnessAndVirtualBass(double sampleRate) noexcept
        : sampleRate_{sampleRate} {
        initFilters();
    }

    void setVolumeAttenuation(double attenuationDb) noexcept {
        attenuationDb_ = std::clamp(attenuationDb, -60.0, 0.0);
        updateLoudnessContours();
    }

    void setVirtualBassAmount(double amount) noexcept {
        bassAmount_ = std::clamp(amount, 0.0, 2.0);
    }

    [[nodiscard]] double process(double in) noexcept {
        const double sub =
subExtractionLPF2_.process(subExtractionLPF1_.process(in));
        const double mainClean =
subRemovalHPF2_.process(subRemovalHPF1_.process(in));

        constexpr double vThresh = 0.12;
        const double norm = std::clamp(sub / vThresh, -1.0, 1.0);
        const double h2 = 2.0 * norm * norm - 1.0;
        const double h3 = 4.0 * norm * norm * norm - 3.0 * norm;
        const double synthHarmonics = (0.65 * h2 + 0.35 * h3) *
vThresh * bassAmount_;

        const double shapedHarmonics =
harmonicBPF_.process(synthHarmonics);

        allpassDelayBuffer_[delayIdx_] = mainClean;
        constexpr size_t kDelaySamples = 44;
        const size_t readIdx = (delayIdx_ + kDelayBufferMask + 1 -
kDelaySamples) & kDelayBufferMask;
        const double alignedMain = allpassDelayBuffer_[readIdx];
        delayIdx_ = (delayIdx_ + 1) & kDelayBufferMask;

        const double summed = alignedMain + shapedHarmonics;
```

```
        const double s1 = lowShelfBiquad_.process(summed);
        const double s2 = highShelfBiquad_.process(s1);
        return midPeakBiquad_.process(s2);
    }

private:
    void initFilters() noexcept {
        const auto lpfCoeffs = Biquad::makeLowShelf(sampleRate_, 45.0,
-12.0, 0.707);
        subExtractionLPF1_.setCoefficients(lpfCoeffs);
        subExtractionLPF2_.setCoefficients(lpfCoeffs);

        const auto hpfCoeffs = Biquad::makeHighShelf(sampleRate_,
45.0, -24.0, 0.707);
        subRemovalHPF1_.setCoefficients(hpfCoeffs);
        subRemovalHPF2_.setCoefficients(hpfCoeffs);

        harmonicBPF_.setCoefficients(Biquad::makePeak(sampleRate_,
90.0, 0.0, 1.2));
        updateLoudnessContours();
    }

    void updateLoudnessContours() noexcept {
        const double lowGain = -0.38 * attenuationDb_;
        const double highGain = -0.11 * attenuationDb_;
        const double midGain = +0.05 * attenuationDb_;


lowShelfBiquad_.setCoefficients(Biquad::makeLowShelf(sampleRate_,
85.0, lowGain, 0.707));

highShelfBiquad_.setCoefficients(Biquad::makeHighShelf(sampleRate_,
9000.0, highGain, 0.707));
        midPeakBiquad_.setCoefficients(Biquad::makePeak(sampleRate_,
3200.0, midGain, 1.4));
    }

    double sampleRate_;
    double attenuationDb_{0.0};
    double bassAmount_{1.0};

    Biquad subExtractionLPF1_{}, subExtractionLPF2_{};
    Biquad subRemovalHPF1_{}, subRemovalHPF2_{};
    Biquad harmonicBPF_{};
    Biquad lowShelfBiquad_{}, highShelfBiquad_{}, midPeakBiquad_{};

    static constexpr size_t kDelayBufferSize = 128;
```

```
    static constexpr size_t kDelayBufferMask = kDelayBufferSize - 1;
    std::array<double, kDelayBufferSize> allpassDelayBuffer_{};
    size_t delayIdx_{0};
};

} // namespace Audiophile::DSP

#endif // AUDIOPHILE_DSP_PIPELINE_HPP
```

# Technical Validation and Verification Specifications

Evaluating and validating this processing pipeline involves verifying each stage against theoretical and acoustic benchmarks. The verification framework couples frequency sweeps, dynamic code exercising, and time-domain phase measurements to ensure mathematical accuracy and numerical stability.

## Koren Triode, Pentode, and Magnetic Transformer Validation

The vacuum tube models are evaluated by checking simulated DC operating curves against published RCA/Mullard tube datasheets. Driving the control grid voltage v_{gk} across a range from -5.0\text{ V} to 0.0\text{ V} while sweeping plate voltage v_{pk} from 0\text{ V} to 400\text{ V} establishes static plate current curves, which must track within 1.5\% of empirical benchmarks. Dynamic harmonic generation is verified by injecting a 1.0\text{ kHz} sinusoidal test tone at -6.0\text{ dBFS}. Fast Fourier Transform (FFT) analysis of the output confirms monotonic harmonic rolloff, ensuring dominant second-order distortion (H_2 \approx -38\text{ dBc}), controlled third-order distortion (H_3 \approx -56\text{ dBc}), and higher-order terms suppressed below -78\text{ dBc}.
Output transformer saturation is tested by applying low-frequency test tones between 10\text{ Hz} and 20\text{ Hz} at +3\text{ dBFS}. Lissajous projections of magnetic flux density B versus field intensity H must display characteristic minor-loop hysteresis spreading. Concurrently, the collapse of differential permeability must elevate the third harmonic by more than +18\text{ dB} relative to midband reference levels, validating the physical Jiles-Atherton differential model.

## Virtual R2R NOS DAC Validation

The Non-Oversampling DAC stage undergoes spectral verification to ensure proper aperture compensation and realistic resistor network non-linearities. A logarithmic sine sweep from 20\text{ Hz} to 20\text{ kHz} at baseband sampling rate f_s = 44.1\text{ kHz} measures output response flatness. With the 5-tap linear-phase inverse-sinc filter engaged, the passband amplitude must remain flat within \pm 0.08\text{ dB} up to 18\text{ kHz}, successfully compensating for the -3.18\text{ dB} uncompensated sinc aperture droop at that frequency.
Static linearity is validated by applying a full-scale 24-bit ramp across the major carry transition code boundaries (0x7FFFFF \to 0x800000). With resistor mismatch tolerance configured to \sigma_R = 1.0 \times 10^{-4}, the measured Differential Non-Linearity spike must remain within 0.8\text{ LSB} \le \text{DNL} \le 1.5\text{ LSB}. This response replicates the low-level step discontinuities typical of discrete multi-bit R2R resistor networks without causing catastrophic

monotonic inversion.

## Sony DC Phase Linearizer Verification

Phase response characteristics are verified using an unwrapped cross-spectral phase transfer function measured between 5\text{ Hz} and 1\text{ kHz}. The processor must maintain unit magnitude gain (\Delta \vert{}H(f)\vert{} = 0.000\text{ dB}) across all audible frequencies while introducing specific low-frequency phase shifts. Under the Type A Standard preset, the phase response must demonstrate a phase advance of +38.2^\circ \pm 1.0^\circ at 20\text{ Hz}, returning smoothly toward 0.0^\circ (\pm 0.2^\circ) above 150\text{ Hz}. Under the Type B Standard preset, the phase response must shift to +62.4^\circ \pm 1.5^\circ at 20\text{ Hz}, matching the extended phase lag typical of transformer- and capacitor-coupled tube stages.

| Presets and Stages | Verification Frequency | Magnitude Tolerance | Target Phase Shift | Pass / Fail Metric |
|---|---|---|---|---|
| **DC Phase Type A Std** | 20.0\[span_55](start_span)[span_55](end_span)text{ Hz} | 0.000\text{ dB} | +38.2^\circ | Error < \pm 1.0^\circ |
| **DC Phase Type B Std** | 20.0\text{ Hz} | 0.000\text{ dB} | +62.4^\circ | Error < \pm 1.5^\circ |
| **NOS DAC Compensation** | 18.0\text{ kHz} | +3.18\text{ dB} boost | 0.0^\circ (Linear) | Baseband ripple < \pm 0.08\text{ dB} |
| **Missing Fundamental LR4** | 45.0\text{ [span_57](start_span)[span_57](end_span)Hz} | -6.02\text{ dB} (Crossover) | -180.0^\circ | High-pass rejection > 24.0\text{ dB} at 25\text{ Hz} |
| **CIFF SDM Modulator** | 1.0\text{ kHz} | 0.0\text{ dB} | Unwrapped | In-band SNR > 128.0\text{ dB} |

## 5th-Order CIFF DSD256 Modulator Validation

The delta-sigma reconstruction loop is tested with a 1.0\text{ kHz} sinusoidal input at -10\text{ dBFS} running at an oversampled clock rate of 11.2896\text{ MHz}. Computing a 65,536-point Blackman-Harris windowed FFT over the audio band (20\text{ Hz} to 24\text{ kHz}) confirms that in-band quantization noise remains suppressed, yielding a baseband SNR greater than 128\text{ dB}. In addition, observing the shaped noise floor reveals two distinct transmission zeroes placed at f_1 \approx 12.9\text{ kHz} and f_2 \approx 21.7\text{ kHz}, verifying that the local feedback resonator coefficients g_1 and g_2 match their designed values.

## ISO 226:2023 Loudness and Virtual Sub-Bass Verification

The equal-loudness compensation filter is evaluated by sweeping the digital attenuation control through -10\text{ dBFS}, -30\text{ dBFS}, and -50\text{ dBFS}. The low-shelf and high-shelf filters must track the updated ISO 226:2023 equal-loudness curves within \pm 0.5\text{ dB} across the 20\text{ Hz} to 12.5\text{ kHz} band.

The missing-fundamental sub-bass generator is validated by applying an isolated 30\text{ Hz} sinusoidal test tone. Spectral analysis must demonstrate that the physical 30\text{ Hz} fundamental is attenuated by more than 24\text{ dB} in the primary output path, while the

synthesized second harmonic (60\text{ Hz}) and third harmonic (90\text{ Hz}) are regenerated in their place. Harmonic intermodulation sidebands outside integer multiples must remain below -60\text{ dBc}. Verifying that the group delay of the clean signal path matches the harmonic path ensures that low-frequency transients fuse into an acoustically coherent bass image.

## Works cited


1. Finding SPICE tube model parameters - Norman Koren, https://www.normankoren.com/Audio/Tube_params.html 2. How Tube Saturation Is Modeled: The Koren Equations Explained, https://kaluaudio.com.br/learn/koren-tube-model-explained/ 3. (PDF) Deep Learning-Based Wave Digital Modeling of Rate, https://www.researchgate.net/publication/369088792_Deep_Learning-Based_Wave_Digital_Modeling_of_Rate-Dependent_Hysteretic_Nonlinearities_for_Virtual_Analog_Applications 4. Analog Audio Circuit Simulation Framework | PDF - Scribd, https://www.scribd.com/document/391624958/Piero-Rivera-Benois-Master-Thesis 5. Circuit modeling studies related to guitars and audio processing, https://aaltodoc.aalto.fi/server/api/core/bitstreams/b1457341-8fc7-4d01-9c96-2412d24d1e19/content 6. WM1AM2 Walkman® Digital Media Player - Sony Asia Pacific, https://www.sony-asia.com/electronics/walkman/nw-wm1am2 7. NW-WM1AM2 Specifications | Walkman - Sony Asia Pacific, https://www.sony-asia.com/electronics/walkman/nw-wm1am2/specifications 8. FM Stereo/ FM-AM Receiver - Sony, https://www.sony.com/electronics/support/res/manuals/2320/23207451M.pdf 9. NW-A55/A55HN/A55L/A56/A56HN/A57 | Help Guide, https://helpguide.sony.net/dmp/nwa50/v1/en/contents/TP0002342874.html 10. SONY NW-WM1Z M2 / WM1A M2 | Page 869 - Head-Fi.org, https://www.head-fi.org/threads/sony-nw-wm1z-m2-wm1a-m2.962054/page-869 11. Sony NW-WM1AM2 Walkman® High-resolution portable digital, https://www.crutchfield.com/p_158NW1AM2/Sony-NW-WM1AM2-Walkman.html 12. SONY NW-WM1Z M2 / WM1A M2 | Page 860 - Head-Fi, https://www.head-fi.org/threads/sony-nw-wm1z-m2-wm1a-m2.962054/page-860 13. Review: Sony WM1A Walkman - Reform - Headfonia, https://www.headfonia.com/review-sony-wm1a-walkman/2/ 14. Sony NW-WM1AM2 Review - Mobileaudiophile, https://mobileaudiophile.com/daps/sony-nw-wm1am2-review/ 15. NW-ZX300/ZX300A | Help Guide | Adjusting the sound quality settings, https://helpguide.sony.net/dmp/nwzx300/v1/en/contents/TP0001449051.html 16. Sony NW-WM1Z Portable Player and MDR-Z1R Headphones, https://www.theabsolutesound.com/articles/sony-nw-wm1z-portable-player-and-mdr-z1r-headphones/ 17. Low-Volume Mixing and the Case for ISO 226:2023, https://giovannicordova.com/blog/low-volume-mixing-iso-226-2023-equal-loudness-compensation 18. ISO 226:2023 Equal-Loudness Contours Analysis | PDF - Scribd, https://www.scribd.com/document/863361703/ISO-226-2023 19. Revision of ISO 226 "Normal Equal-Loudness-Level Contours" from, https://www.jstage.jst.go.jp/article/ast/45/1/45_e23.66/_article 20. ISO 226:2023 | iTeh Standards, https://cdn.standards.iteh.ai/samples/83117/6afa5bd94e0e4f32812c28c3b0a7b8ac/ISO-226-2023.pdf 21. 10 Best Enhancer Plugins in 2026 (Stereo, Bass, Vocals & More), https://pluginreviewlab.com/best-enhancer-plugins/ 22. MaxxBass: Enhancing Loudspeaker

Bass | PDF - Scribd, https://www.scribd.com/document/396601541/10-1-1-524-4796-pdf 23. University of Southampton Research Repository ePrints Soton, https://eprints.soton.ac.uk/id/eprint/368484/1/prw_phd.pdf